

TankSight Master Architecture Plan

Inspection Reporting & Client Portal Prototype

Komlan Kouhiko

2026-07-07

Contents

TankSight — Inspection Reporting & Client Portal Prototype	2
Codex Master Architecture Prompt	2
TankSight — Inspection Reporting & Client Portal Prototype	2
1. Project Goal	2
2. Tech Stack	3
3. High-Level Architecture	3
4. Core Architectural Layers	3
4.1 Presentation Layer	3
4.2 Component Layer	4
4.3 API Layer	4
4.4 Application Service Layer	5
4.5 Domain Layer	6
4.6 Repository/Data Access Layer	7
5. Full Folder Structure	8
6. Data Model	10
7. Seed Data Requirements	12
8. Data Flow	13
9. API Contracts	13
GET /api/client/summary	13
GET /api/sites/[siteSlug]	14
GET /api/tanks/[tankSlug]	14
GET /api/inspection-runs/[inspectionRunId]/measurements	14
GET /api/inspection-runs/[inspectionRunId]/report	15
10. Page Requirements	15
/login	15
/client/dashboard	15
/client/sites/[siteSlug]	15
/client/tanks/[tankSlug]	15
/client/tanks/[tankSlug]/report-preview	16
/inspector/dashboard	16
11. PDF Architecture	16

12. UI/Design System	16
13. README Requirements	17
14. Build Order	17
15. Acceptance Criteria	18
16. Limitations to State	18
18. Login Portal / Demo Access Requirements	19
Routes	19
Demo Login Behavior	19
Access Scoping Rules	19
Login UI Requirements	20
Production Auth Note	20
19. Seed Data Export / Import Requirements	20
Seed Data Location	20
Exportable Seed Files	20
Export Script Requirements	21
Optional CSV Export	21
README Requirement	21
20. Final Demo Polish / 10-10 Requirements	22
Deployment Readiness	22
Demo Walkthrough and Screenshots	22
Sample Processed Robot Output Import File	23
Report Download / Audit Log Concept	23
Loading, Error, and Empty States	24
In-App Production Path Section	24
Final 10-10 Review Flow	24
21. Final Output	24

TankSight — Inspection Reporting & Client Portal Prototype

Codex Master Architecture Prompt

You are a senior full-stack engineer. Build a polished aptitude-test prototype called:

TankSight — Inspection Reporting & Client Portal Prototype

1. Project Goal

Build a focused full-stack prototype for an inspection reporting/client portal product.

The company is building a robot for automated oil tank floor inspections using processed PAUT thickness/corrosion data. This prototype should demonstrate how processed inspection data becomes:

1. A client portal
2. Tank/site/inspection history
3. Interactive tank floor heatmaps
4. Region-level corrosion/thickness summaries
5. Trend comparison across inspection runs
6. Downloadable client-ready PDF inspection reports

Do not build raw ultrasonic signal processing. The data arrives already processed.
This is not a marketing site. This is a data/reporting/client-portal prototype.
The final result should feel like a mini version of the company's future product.

2. Tech Stack

Use:

- Next.js App Router
 - React
 - TypeScript
 - Tailwind CSS
 - Next.js Route Handlers for APIs
 - Seeded mock data structured like MongoDB collections
 - SVG for the tank floor heatmap
 - Recharts or Chart.js for trend charts
 - @react-pdf/renderer for PDF generation
 - No real auth, but simulate role/client scoping
 - No real database, but create a clean repository layer that can later be replaced by MongoDB
-

3. High-Level Architecture

Use this layered architecture:

```
UI Routes
↓
Page-level server components / client components
↓
Feature components
↓
API route handlers
↓
Application services
↓
Domain/metric functions
↓
Repository/data-access layer
↓
Seeded mock collections
```

The goal is to keep the app organized like a real production system.
Do not put business calculations directly in pages.
Do not put seed-data filtering directly in components.
Do not mix PDF logic with UI components.

4. Core Architectural Layers

4.1 Presentation Layer

Responsible for routes and page composition.

Routes:

```
/login
  Demo login portal / role-based demo access

/
  Redirect or link to /login

/client/dashboard
  Client portal dashboard for Acme Energy

/client/sites/[siteSlug]
  Site detail page with tanks and latest inspection status

/client/tanks/[tankSlug]
  Tank inspection detail page

/client/tanks/[tankSlug]/report-preview
  Web preview of the inspection report

/inspector/dashboard
  Internal inspector dashboard
```

Pages should load data through application services or repository functions, not directly from seed arrays.

4.2 Component Layer

Reusable UI and inspection-specific components.

Components must be organized by responsibility:

```
components/layout
components/ui
components/dashboard
components/inspection
components/report
```

Important components:

```
AppShell
Header
MetricCard
StatusBadge
DataTable
TankFloorHeatmap
RegionDetailsPanel
RegionBreakdownTable
InspectionTrendChart
FindingsList
ExecutiveSummaryCard
ReportActions
ReportPreview
```

4.3 API Layer

Use Next.js Route Handlers.

API route handlers should call services, not seed data directly.

Required API endpoints:

```
GET /api/client/summary
GET /api/inspector/summary
GET /api/sites/[siteSlug]
GET /api/tanks/[tankSlug]
GET /api/inspection-runs/[inspectionRunId]
GET /api/inspection-runs/[inspectionRunId]/measurements
GET /api/inspection-runs/[inspectionRunId]/report
```

These endpoints should return structured JSON except the report endpoint, which returns a PDF.

4.4 Application Service Layer

Create a service layer for use cases.

Services should live in:

```
src/lib/services/
```

Required services:

```
client-dashboard-service.ts
inspector-dashboard-service.ts
site-service.ts
tank-service.ts
inspection-service.ts
report-service.ts
```

Responsibilities:

client-dashboard-service.ts

Builds the Acme client dashboard.

Returns:

```
{
  client,
  summaryCards,
  latestInspection,
  siteTankRows
}
```

inspector-dashboard-service.ts

Builds the inspector dashboard.

Returns:

```
{
  totals,
  recentInspections,
  reportQueue,
  tanksRequiringReview
}
```

site-service.ts

Gets a site and its tank summaries.

Returns:

```
{
  site,
  tanks: TankSummary[]
}
```

tank-service.ts

Gets tank detail, latest inspection, metrics, findings, region summaries, trends.

Returns:

```
{
  client,
  site,
  tank,
  latestRun,
  previousRun,
  metrics,
  regionSummaries,
  findings,
  trendData,
  executiveSummary
}
```

inspection-service.ts

Handles inspection-run-level data.

Returns measurement summaries, paginated measurement cells, and trend metrics.

report-service.ts

Prepares report data used by both:

```
/client/tanks/[tankSlug]/report-preview
/api/inspection-runs/[inspectionRunId]/report
```

Important: the report preview and PDF should use the same report data builder.

4.5 Domain Layer

This is where inspection calculations live.

Create:

```
src/lib/domain/
```

Files:

```
inspection-metrics.ts
severity.ts
region.ts
formatting.ts
trend.ts
```

severity.ts

Responsible for mock severity thresholds.

Use prototype thresholds:

```
critical: thickness <= 0.190 OR metal loss >= 24
concern: thickness <= 0.210 OR metal loss >= 16
monitor: thickness <= 0.230 OR metal loss >= 8
ok: above monitor threshold
```

Add clear comments:

```
// Prototype thresholds only.
// These are not API 653-certified calculations.
```

region.ts

Responsible for assigning region based on radius/angle.

Regions:

```
center
north
south
east
west
annular
```

inspection-metrics.ts

Required functions:

```
calculateMinThickness(cells)
calculateAverageThickness(cells)
calculateMetalLossPercent(thickness, nominalThickness)
countSeverity(cells)
groupCellsByRegion(cells)
buildRegionSummaries(cells, findings)
calculateRemainingLifeEstimate(currentRun, previousRun, currentCells, previousCells, tank)
buildExecutiveSummary(tank, currentRun, previousRun, cells, findings)
```

trend.ts

Required functions:

```
buildInspectionTrendData(tankId)
calculateRunMetrics(run, cells)
compareInspectionRuns(currentRun, previousRun)
```

4.6 Repository/Data Access Layer

Create:

```
src/lib/data/repository.ts
```

This layer simulates database access.

It must be the only place that directly reads from seeded collections.

Required functions:

```

getClientById(clientId)
getAllClients()
getSitesForClient(clientId)
getSiteBySlug(siteSlug, clientId?)
getTanksForSite(siteId, clientId?)
getTankBySlug(tankSlug, clientId?)
getInspectionRunsForTank(tankId, clientId?)
getLatestInspectionRunForTank(tankId, clientId?)
getInspectionRunById(inspectionRunId, clientId?)
getMeasurementCellsForRun(inspectionRunId, filters?)
getFindingsForRun(inspectionRunId)
getClientDashboardSummary(clientId)
getInspectorDashboardSummary()

```

Client scoping rule:

When `clientId` is provided, every repository function must filter by `clientId`.
When `clientId` is omitted, the function behaves like an inspector/admin query.

Add comments explaining production MongoDB indexes:

```

/*
Recommended MongoDB indexes:

clients:
{ _id: 1 }

sites:
{ clientId: 1, slug: 1 }

tanks:
{ clientId: 1, siteId: 1, slug: 1 }

inspectionRuns:
{ clientId: 1, tankId: 1, inspectedAt: -1 }

measurementCells:
{ inspectionRunId: 1, region: 1, severity: 1 }
{ clientId: 1, siteId: 1, tankId: 1, inspectionRunId: 1 }
{ inspectionRunId: 1, x: 1, y: 1 }

findings:
{ inspectionRunId: 1, severity: 1 }

reportJobs:
{ inspectionRunId: 1, status: 1, createdAt: -1 }
*/

```

5. Full Folder Structure

Create this exact structure:

```

tanksight-prototype/
  README.md
  package.json
  next.config.ts
  tsconfig.json
  tailwind.config.ts
  postcss.config.js

  src/
    app/
      globals.css
      layout.tsx
      page.tsx

```



```
client/
  dashboard/
    page.tsx

  sites/
    [siteSlug]/
      page.tsx

  tanks/
    [tankSlug]/
      page.tsx

    report-preview/
      page.tsx

inspector/
  dashboard/
    page.tsx

api/
  client/
    summary/
      route.ts

  inspector/
    summary/
      route.ts

  sites/
    [siteSlug]/
      route.ts

  tanks/
    [tankSlug]/
      route.ts

  inspection-runs/
    [inspectionRunId]/
      route.ts

    measurements/
      route.ts

    report/
      route.ts

components/
  layout/
    AppShell.tsx
    Header.tsx
    RoleBadge.tsx

  ui/
    MetricCard.tsx
    StatusBadge.tsx
    DataTable.tsx
    SectionCard.tsx
    EmptyState.tsx

dashboard/
  ClientDashboard.tsx
  InspectorDashboard.tsx
  SiteTankTable.tsx
  ReportQueueTable.tsx

inspection/
  TankHeader.tsx
  TankMetricsGrid.tsx
  ExecutiveSummaryCard.tsx
  TankFloorHeatmap.tsx
  RegionDetailsPanel.tsx
```

```

    RegionBreakdownTable.tsx
    InspectionTrendChart.tsx
    FindingsList.tsx
    ReportActions.tsx

    report/
      ReportPreview.tsx
      ReportSection.tsx

    lib/
      types.ts

    data/
      seed.ts
      generate-measurements.ts
      repository.ts

    domain/
      severity.ts
      region.ts
      inspection-metrics.ts
      trend.ts
      formatting.ts

    services/
      client-dashboard-service.ts
      inspector-dashboard-service.ts
      site-service.ts
      tank-service.ts
      inspection-service.ts
      report-service.ts

    pdf/
      InspectionReportDocument.tsx
      render-report.ts

    constants/
      demo.ts

```

6. Data Model

Create all types in:

```
src/lib/types.ts
```

Use these types:

```

export type UserRole = "client" | "inspector";

export type Severity = "ok" | "monitor" | "concern" | "critical";

export type InspectionStatus = "healthy" | "monitor" | "action_recommended";

export type ReportStatus = "draft" | "ready";

export type Region =
  | "center"
  | "north"
  | "south"
  | "east"
  | "west"
  | "annular";

export type Client = {
  id: string;
  name: string;

```

```

};

export type User = {
  id: string;
  name: string;
  role: UserRole;
  clientId?: string;
};

export type Site = {
  id: string;
  clientId: string;
  slug: string;
  name: string;
  location: string;
};

export type Tank = {
  id: string;
  clientId: string;
  siteId: string;
  slug: string;
  tankNumber: string;
  diameterFeet: number;
  product: string;
  nominalThicknessInches: number;
};

export type InspectionRun = {
  id: string;
  clientId: string;
  siteId: string;
  tankId: string;
  inspectedAt: string;
  method: "PAUT robotic tank floor scan";
  status: InspectionStatus;
  reportStatus: ReportStatus;
};

export type MeasurementCell = {
  id: string;
  clientId: string;
  siteId: string;
  tankId: string;
  inspectionRunId: string;
  x: number;
  y: number;
  radius: number;
  angle: number;
  region: Region;
  thicknessInches: number;
  nominalThicknessInches: number;
  metalLossPercent: number;
  severity: Severity;
};

export type Finding = {
  id: string;
  inspectionRunId: string;
  region: Region;
  severity: Severity;
  title: string;
  description: string;
  recommendation: string;
};

export type RegionSummary = {
  region: Region;
  readingCount: number;
  minThickness: number;
  avgThickness: number;
  maxMetalLossPercent: number;
};

```

```

    severity: Severity;
    recommendation: string;
};

export type InspectionMetrics = {
    minThickness: number;
    avgThickness: number;
    nominalThickness: number;
    criticalCells: number;
    monitorCells: number;
    concernCells: number;
    okCells: number;
    estimatedRemainingLifeYears: number | null;
    recommendation: string;
};

export type TrendPoint = {
    inspectedAt: string;
    minThickness: number;
    avgThickness: number;
    criticalCells: number;
};

export type ReportData = {
    client: Client;
    site: Site;
    tank: Tank;
    currentRun: InspectionRun;
    previousRun: InspectionRun | null;
    metrics: InspectionMetrics;
    regionSummaries: RegionSummary[];
    findings: Finding[];
    trendData: TrendPoint[];
    executiveSummary: string;
};

```

7. Seed Data Requirements

Create seed data in:

```
src/lib/data/seed.ts
```

Create deterministic mock data.

Required seed data:

```

3 clients
3 sites for Acme Energy
8 tanks for Acme Energy
14 inspection runs across Acme Energy
2 inspection runs for TK-104–
300800 measurement cells for TK-104 2024 inspection–
300800 measurement cells for TK-104 2026 inspection–
35 findings for TK-104 2026 inspection

```

Primary demo objects:

```

Client: Acme Energy
Site: Sterling Terminal
Tank: TK-104
Product: Diesel
Diameter: 120 ft
Nominal Thickness: 0.250 in
Inspection Date: June 18, 2026
Method: PAUT robotic tank floor scan
Status: Action Recommended

```

Create a deterministic measurement generator in:

```
src/lib/data/generate-measurements.ts
```

Generator requirements:

- Generate x/y points inside a circular tank floor
 - Calculate radius and angle
 - Assign region from radius/angle
 - Most cells should be OK
 - Add corrosion cluster in northeast annular ring
 - Make 2026 worse than 2024
 - Do not use unstable `Math.random()` unless seeded/deterministic
 - Data must be the same every page refresh
-

8. Data Flow

The main tank detail page should follow this flow:

```
/client/tanks/tk-104 page
↓
tank-service.getTankDetail("tk-104", "client_acme")
↓
repository.getTankBySlug()
repository.getLatestInspectionRunForTank()
repository.getMeasurementCellsForRun()
repository.getFindingsForRun()
↓
domain functions calculate metrics, summaries, trends
↓
page renders:
  TankHeader
  TankMetricsGrid
  ExecutiveSummaryCard
  TankFloorHeatmap
  RegionBreakdownTable
  InspectionTrendChart
  FindingsList
  ReportActions
```

The PDF flow should be:

```
/api/inspection-runs/[inspectionRunId]/report
↓
report-service.getInspectionReportData(inspectionRunId, clientId?)
↓
repository loads client/site/tank/run/cells/findings
↓
domain functions build metrics/summaries/trends/executive summary
↓
InspectionReportDocument renders PDF
↓
route returns application/pdf
```

9. API Contracts

GET /api/client/summary

Returns:

```

{
  client: Client;
  cards: {
    label: string;
    value: string | number;
    helper?: string;
  }[];
  latestInspection: {
    siteName: string;
    tankNumber: string;
    inspectedAt: string;
    status: InspectionStatus;
    tankSlug: string;
  };
  rows: {
    siteName: string;
    tankNumber: string;
    product: string;
    lastInspection: string;
    minThickness: number;
    avgThickness: number;
    severity: InspectionStatus;
    reportStatus: ReportStatus;
    tankSlug: string;
  }[];
}

```

GET /api/sites/[siteSlug]

Returns:

```

{
  site: Site;
  tanks: {
    tank: Tank;
    latestRun: InspectionRun | null;
    metrics: InspectionMetrics | null;
  }[];
}

```

GET /api/tanks/[tankSlug]

Returns:

```

{
  client: Client;
  site: Site;
  tank: Tank;
  latestRun: InspectionRun;
  previousRun: InspectionRun | null;
  metrics: InspectionMetrics;
  regionSummaries: RegionSummary[];
  findings: Finding[];
  trendData: TrendPoint[];
  executiveSummary: string;
}

```

GET /api/inspection-runs/[inspectionRunId]/measurements

Query params:

```

region optional
severity optional
page optional
limit optional

```

Returns:

```
{
  page: number;
  limit: number;
  total: number;
  cells: MeasurementCell[];
}
```

GET /api/inspection-runs/[inspectionRunId]/report

Returns PDF with headers:

```
Content-Type: application/pdf
Content-Disposition: inline; filename="TK-104-inspection-report.pdf"
```

10. Page Requirements

/login

Demo login portal / role selection page.

Must include:

```
TankSight
Inspection Reporting & Client Portal Prototype
Processed inspection data → portal → heatmap → report
View as Client
View as Inspector
```

/client/dashboard

Must include:

```
Acme Energy dashboard
Summary cards
Latest inspection card
Site/tank table
```

/client/sites/[siteSlug]

Must include:

```
Site header
Tank list table
Download report action for TK-104
```

/client/tanks/[tankSlug]

This is the showcase page.

Must include:

```
Tank header
Metric cards
Executive summary
Interactive tank floor heatmap
Region details side panel
```

```
Region breakdown table
Inspection trend chart
Findings list
Download PDF button
Report preview link
```

/client/tanks/[tankSlug]/report-preview

Must include a web report preview using the same ReportData shape used by the PDF.

/inspector/dashboard

Must include:

```
Internal inspection overview
Recent inspections
Report queue
Tanks requiring review
```

11. PDF Architecture

PDF files:

```
src/lib/pdf/InspectionReportDocument.tsx
src/lib/pdf/render-report.ts
```

Use @react-pdf/renderer.

The report should include:

```
Cover/header
Client/site/tank details
Inspection date/method/status
Executive summary
Thickness summary
Region breakdown table
Corrosion findings
Trend comparison
Simplified heatmap or region severity summary
Recommendations
Disclaimer
Footer/page number if practical
```

Add disclaimer:

```
Prototype uses mock processed PAUT-style data for demonstration only.
This is not an API 653-certified engineering report.
```

The PDF should be professional and consistent.

12. UI/Design System

Use Tailwind.

Recommended design tokens:

Background: slate-950 or slate-100 depending on section
Cards: white
Text: slate-900
Muted text: slate-500
Borders: slate-200
Primary: blue-700 or cyan-700
Critical: red-600
Concern: orange-500
Monitor: amber-400
OK: emerald-600

The interface should feel:

industrial
technical
professional
serious
client-ready

Not playful. Not colorful SaaS fluff.

13. README Requirements

Create a strong README with:

Overview
What TankSight demonstrates
Demo flow
Tech stack
How to run locally
What is mocked
Data model
Architecture
API routes
Report-generation pipeline
Role/client scoping
MongoDB production mapping
How this maps to the role
Production next steps
Limitations/disclaimer

Include this section exactly:

```
## How this maps to the role

- Client portal: implemented
- Tank/site/inspection history: implemented
- Processed PAUT-style measurement data: mocked but structured
- Tank floor heatmap: implemented
- Region breakdown: implemented
- Trend comparison: implemented
- PDF report generation: implemented
- Backend API shape: implemented with Next.js Route Handlers
- Role/client scoping: demonstrated through client vs inspector views
- MongoDB-ready data model: documented with collection-style seed data and recommended indexes
- Report job pipeline: simulated/documented
```

14. Build Order

Build in this order:

1. Scaffold Next.js + TypeScript + Tailwind
2. Create folder structure
3. Create shared types
4. Create deterministic mock measurement generator
5. Create seed data
6. Create repository layer
7. Create domain calculation functions
8. Create service layer
9. Create reusable UI components
10. Build entry role-switch page
11. Build client dashboard
12. Build site detail page
13. Build tank detail page
14. Build SVG tank floor heatmap
15. Build region breakdown table
16. Build trend chart
17. Build report preview page
18. Build PDF generation route
19. Build inspector dashboard
20. Create API routes
21. Write README
22. Final polish pass

15. Acceptance Criteria

The prototype is complete when:

```
npm install works
npm run dev works
TypeScript compiles
User can open /login and choose client role
User can open client dashboard
User can open Sterling Terminal
User can open TK-104
Heatmap renders from measurement data
Region table renders from measurement data
Trend chart compares 2024 vs 2026
Report preview works
PDF download works
Inspector dashboard works
README explains the architecture clearly
No raw ultrasonic/DSP claims are made
No API 653 certification claims are made
Code is organized and readable
```

16. Limitations to State

Use this language in the README and, where appropriate, in the UI:

```
This prototype uses mock processed PAUT-style measurement data for demonstration only. It does not perform raw ultrasonic signal processing, certified API 653 calculations, or engineering review. It demonstrates the software architecture for organizing inspection data, visualizing tank floor conditions, and generating client-facing inspection reports.
```

18. Login Portal / Demo Access Requirements

The prototype must include a lightweight login/demo access portal, not just plain navigation buttons. This does not need full production authentication, but it should visually and architecturally demonstrate how the future product would handle role-based access.

Routes

Create:

```
/login
  Demo login portal

/
  Redirect or link to /login
```

Demo Login Behavior

The login page should show two demo accounts:

```
Client Demo
Email: client@acme.example
Role: Client
Access: Acme Energy only

Inspector Demo
Email: inspector@tanksight.example
Role: Inspector
Access: All clients, all sites, all tanks, all inspection runs
```

Use simple buttons:

```
Continue as Client
Continue as Inspector
```

No password required for the aptitude prototype.

When the user selects a role, store the selected role/client in one of these simple prototype-safe ways:

```
URL query param
localStorage
cookie
server action with demo session cookie
```

Recommended simple approach:

```
Use a demo cookie:
demoRole=client or inspector
demoClientId=client_acme for client users
```

Then route:

```
Client → /client/dashboard
Inspector → /inspector/dashboard
```

Access Scoping Rules

Client mode:

```
Can only see Acme Energy data.
Cannot see other seeded clients.
All repository/service calls must pass clientId = "client_acme".
```

Inspector mode:

Can see all clients, sites, tanks, inspection runs, report queue, and tanks requiring review.
Repository/service calls may omit clientId.

Login UI Requirements

The login portal should feel like an industrial client portal login, not a playful demo screen.

Include:

TankSight logo/name
Inspection Reporting & Client Portal
Demo access cards
Short note: "Prototype demo access – no production authentication enabled"
Role badges
Clear access descriptions

Production Auth Note

Add this to README:

The prototype uses demo role selection instead of production authentication. In production, this would be replaced with real authentication, organization membership, role-based permissions, audit logs, and server-side session enforcement.

19. Seed Data Export / Import Requirements

The prototype must include seeded data that can be exported and reviewed.

This is important because the company should be able to inspect the mock PAUT-style data used by the portal, heatmap, trend charts, and PDF report.

Seed Data Location

Seed data should live in:

```
src/lib/data/seed.ts
```

Generated measurement data should come from:

```
src/lib/data/generate-measurements.ts
```

Exportable Seed Files

Create an export script that writes seed data to JSON files.

Create:

```
scripts/export-seed-data.ts
```

The script should export the seeded collections into:

```
seed-export/  
  clients.json  
  users.json  
  sites.json  
  tanks.json
```

```
inspectionRuns.json
measurementCells.json
findings.json
reportJobs.json
```

Add package script:

```
{
  "scripts": {
    "export:seed": "tsx scripts/export-seed-data.ts"
  }
}
```

If `tsx` is not installed, add it as a dev dependency.

Export Script Requirements

The export script should:

```
Create seed-export/ if it does not exist
Write pretty-formatted JSON
Include deterministic generated measurement cells
Print a summary count after export
Never include secrets
```

Example terminal output:

```
Seed export complete:
clients: 3
users: 2
sites: 6
tanks: 14
inspectionRuns: 20
measurementCells: 1200
findings: 5
reportJobs: 2
Output: ./seed-export
```

Optional CSV Export

If time allows, also create CSV exports for measurement cells and region summaries:

```
seed-export/measurementCells.csv
seed-export/regionSummaries-TK-104-2026.csv
```

This is optional, but valuable because inspection and engineering teams often review tabular data.

README Requirement

Add a README section:

```
## Seed Data Export

The app uses deterministic mock processed PAUT-style inspection data. To export the seeded collections
for review:

```bash
npm run export:seed
```

This writes JSON files to `seed-export/`, including clients, sites, tanks, inspection runs, measurement cells, findings, and report jobs.

The exported data can be reviewed directly or used as a starting point for a MongoDB import.

```
MongoDB Import Path
```

Document that the JSON files map directly to future MongoDB collections:

```
```txt
clients.json → clients collection
sites.json → sites collection
tanks.json → tanks collection
inspectionRuns.json → inspectionRuns collection
measurementCells.json → measurementCells collection
findings.json → findings collection
reportJobs.json → reportJobs collection
```

Add a production note:

In production, the seeded data layer would be replaced by MongoDB collections with indexes on `clientId`, `siteId`, `tankId`, `inspectionRunId`, `region`, `severity`, and `inspectedAt`.

20. Final Demo Polish / 10-10 Requirements

These requirements should be included so the prototype feels like a serious engineering submission, not just a coding exercise.

Deployment Readiness

The app should be ready to deploy to Vercel.

Add or verify:

```
next.config.ts exists
npm run build works
No local-only filesystem dependencies in page rendering
PDF route uses Node runtime when required
README includes deployment notes
```

Demo Walkthrough and Screenshots

Add a README section called:

```
## Demo Walkthrough
```

Include the intended review flow:

```
1. Open /login
2. Continue as Client
3. View Acme Energy dashboard
4. Open Sterling Terminal
5. Open TK-104
6. Inspect tank floor heatmap
7. Click the corrosion cluster
8. Review region breakdown and trends
9. Open report preview
10. Download PDF report
11. Run npm run export:seed to review mock data
```

If time allows, add screenshots under:

```
public/demo-screenshots/
```

Recommended screenshots:

```
login.png
dashboard.png
tank-detail-heatmap.png
report-preview.png
pdf-report.png
```

The README should reference these screenshots if they exist.

Sample Processed Robot Output Import File

Add a sample processed robot output file to show how the future robot pipeline could feed the app.

Create:

```
sample-data/processed-paut-output-TK-104-2026.json
```

This file should include a simplified import-ready shape:

```
{
  clientId: "client_acme",
  siteId: "site_sterling_terminal",
  tankId: "tank_tk_104",
  inspectionRunId: "run_tk_104_2026",
  source: "mock_robot_processed_output",
  method: "PAUT robotic tank floor scan",
  measurements: [
    {
      x: 12.4,
      y: -33.8,
      thicknessInches: 0.184,
      nominalThicknessInches: 0.25,
      region: "annular"
    }
  ]
}
```

Add a README note explaining that this is not raw ultrasonic data. It is already-processed measurement data.

Report Download / Audit Log Concept

Add a lightweight audit-log concept in the mock data and README.

Create a type or documented collection concept:

```
export type ReportDownloadLog = {
  id: string;
  clientId: string;
  inspectionRunId: string;
  downloadedByUserId: string;
  downloadedAt: string;
  fileName: string;
};
```

For the prototype, this can be simulated/documented only. In production, report downloads should create immutable audit records.

README should include:

```
## Report Audit Trail Concept
```

In production, every report generation, approval, download, and revision should create an audit record. This prototype documents the audit-log shape but does not persist real audit events.

Loading, Error, and Empty States

Add simple but polished states for:

```
loading data
missing tank
missing site
no inspections found
PDF generation failure
no measurement cells after filter
```

Use reusable components:

```
EmptyState.tsx
ErrorState.tsx optional
LoadingPanel.tsx optional
```

In-App Production Path Section

On the inspector dashboard or report preview page, include a small section titled:

```
What would be productionized next
```

List:

```
MongoDB persistence
Real authentication and organization membership
Redis-backed report-generation queue
Object storage for generated PDFs
CSV/JSON import pipeline for processed robot output
Report approvals and revision history
Audit logs for report downloads
API 653 calculation layer with qualified engineering input
Performance testing for large measurement collections
```

Final 10-10 Review Flow

The prototype should feel strongest if this flow works smoothly:

```
Login as Client→
  See Acme Energy dashboard→
    Open Sterling Terminal→
      Open TK-104→
        See interactive tank floor heatmap→
          Click corrosion cluster→
            See region summary and trend decline→
              Open report preview→
                Download PDF report→
                  Export seed data
```

21. Final Output

When finished, summarize:

```
What was built
How to run it
Key files created
Architecture summary
Known limitations
Production next steps
```